

ФКН ВШЭ
Глубинное обучение 1, 2024/25

Автор конспекта: Даниил Тихонов

27 января 2025 г. в 12:29

Содержание

1. Введение в работу с последовательностями	1
1.1. Последовательности	1
1.2. Формальная постановка	1
1.3. Токенизация	2
1.4. Эмбединги	4
1.5. FastText	6
1.6. TextCNN	7

Лекция #7: Введение в работу с последовательностями

25 ноября

1.1. Последовательности

Чаще всего под последовательностями подразумеваются тексты, но есть и множество других форматов данных, которые можно представить в таком виде:

- Последовательность кадров – видео
- Последовательность звукового давления – музыка
- Последовательность значений во времени – временной ряд

Далее для простоты восприятия будем говорить про тексты. При работе с ними возникают две проблемы, которых не было в прежде рассмотренных задачах.

Во-первых, нейросети привыкли работать с непрерывными значениями, а значит, каждый текст нужно уметь переводить в такое представление. В отличие от картинок, где значения пикселей хоть и являются целыми, но их величина имеет физический смысл, их можно сравнивать. Если же, например, рассмотреть текст как последовательность символов, то обычная кодировка по типу ASCII или Unicode не имеет никакого внутреннего смысла.

Во-вторых, тексты обладают переменной длиной. Сравнивая с теми же картинками, есть множество методов интерполяции изображений в нужный размер, которые сохраняют контент. Масштабирование текста уже не очень очевидно. В случае сжатия текста, возможно, еще и можно придумать адекватные эвристики по типу простого обрезания последовательности по нужному количеству символов, а как увеличить текст совсем непонятно.

Таким образом, главной задачей является придумать разумное кодирование текста и научить модель работать с последовательностями произвольной длины.

1.2. Формальная постановка

Пусть имеется некоторая последовательность $w_1, \dots, w_T$ (например, слова или символы). Далее эта последовательность произвольных сущностей преобразуется в последовательность токенов $t_1, \dots, t_L$, где $t_i \in \text{Vocab}$ – фиксированное множество, называемое **словарем**. Важно заметить, что длина новой последовательности зависит от способа токенизации и может как увеличиться, так и уменьшиться.

Токены все еще являются элементами дискретного множества, поэтому каждому из них нужно поставить в соответствие некоторое информативное векторное представление, называемое **эмбеддингом**: $t_i \mapsto e_i \in \mathbb{R}^d$. Размерность эмбеддингов d является гиперпараметром и фиксируется заранее для всех токенов (обычно $d \in \{64, 128, 256\}$). В дальнейшем эти эмбеддинги и будут подаваться в качестве входа для нейросети.

Есть несколько основных подходов к построению эмбеддингов для токенов:

- Можно взять заранее предобученные на другой задаче с большим объемом данных эмбеддинги и использовать их для оптимизации модели. Это хороший путь, если имеющийся датасет не очень большой, а смежная задача не ортогональна нашей.
- Если же имеется сразу много данных, то эмбеддинги можно обучать как часть модели.

Наконец, получили полный пайплайн обучения, рассмотрим каждый этап подробнее.

Sequence $\longrightarrow$ Tokens $\longrightarrow$ Embeddings $\longrightarrow$ Model

1.3. Токенизация

Рассмотрим некоторые способы токенизации на примере строки на русском языке

На дворе трава, на траве дрова.

- **Character-level** – каждый символ является отдельным токеном (в том числе и все знаки препинания и пробелы):

/Н/а/ /д/в/о/р/е/ /т/р/а/в/а/, /н/а/ /т/р/а/в/е/ /д/р/о/в/а/./

(+) Новые слова легко кодируются.

(+) Известный и очень маленький словарь: мало эмбеддингов $\Rightarrow$ легко оптимизировать.

(-) Теряется структура языка: связь символов, их комбинации и лексический смысл.

(-) Последовательность токенов для большинства текстов будет очень длинной.

- **Word-level** – каждое слово является отдельным токеном (пробелы не считаются, но знаки препинания являются отдельными словами):

/На/дворе/трава/,/на/траве/дрова/./

(+) Сохраняется семантическая информация о языке.

(+) Короткие последовательности токенов.

(-) Очень большой словарь (особенно для языков, где есть окончания или спряжения).

(-) Сложно кодировать новые слова.

Для того, чтобы можно было кодировать слова, которых нет в словаре, вводится специальный токен $\langle \text{UNK} \rangle$, отвечающий за неизвестные слова.

Например, в целях экономии ресурсов словарь должен быть ограничен N словами. Тогда можно взять самые часто встречающиеся $N - K$ слов по обучающей выборке, а остальные заменить данным токеном (K – количество специальных токенов).

- **Subword-level** – токенизируются подслова. Рассмотрим один из примеров:

Byte-pair encoding (BPE).

Построение словаря фиксированного размера N происходит итеративно:

1. Начинаем с посимвольной токенизации.
2. Находим самую часто встречающуюся вместе пару токенов в тексте (порядок важен!).
3. Добавляем конкатенацию самой частой пары в словарь, не удаляя прежние токены.
4. Если текущий размер словаря $< N$, переходим в пункту (2).

Рассмотрим несколько шагов на примере нашей строки (пробелы игнорируются):

Шаг 0:

$$\text{Vocab}_0 = \{H, a, \partial, \text{e}, o, p, e, m, n, ', ', '.\}$$

$$|H/a| \ | \partial/\text{e}/o/p/e| \ |m/p/a/\text{e}/a|,| \ |n/a| \ |m/p/a/\text{e}/e| \ | \partial/p/o/\text{e}/a|.$$

Шаг 1:

$$\text{Vocab}_1 = \{H, a, \partial, \text{e}, o, p, e, m, n, ', ', '.', mp\}$$

$$|H/a| \ | \partial/\text{e}/o/p/e| \ |mp/a/\text{e}/a|,| \ |n/a| \ |mp/a/\text{e}/e| \ | \partial/p/o/\text{e}/a|.$$

Шаг 2:

$$\text{Vocab}_2 = \{H, a, \partial, \text{e}, o, p, e, m, n, ', ', '.', mp, mpa\}$$

$$|H/a| \ | \partial/\text{e}/o/p/e| \ |mpa/\text{e}/a|,| \ |n/a| \ |mpa/\text{e}/e| \ | \partial/p/o/\text{e}/a|.$$

Шаг 3:

$$\text{Vocab}_3 = \{H, a, \partial, \text{e}, o, p, e, m, n, ', ', '.', mp, mpa, mpa\}$$

$$|H/a| \ | \partial/\text{e}/o/p/e| \ |mpa\text{e}/a|,| \ |n/a| \ |mpa\text{e}/e| \ | \partial/p/o/\text{e}/a|.$$

В реальных текстах токены будут разнообразнее и словарь будет состоять из самых часто встречающихся **n-грамм** (последовательностей из n символов).

- (+) Словарь создается динамически на основе данных, а не волевых решениях.
- (+) Все часто встречающиеся комбинации будут объединены в токены и нести семантическую информацию о языке.
- (+) Новые/редкие слова будут токенизироваться, так как все символы есть в словаре.
- (+) Легко контролируемый размер словаря.

На первый взгляд кажется, что все это будет считать долго для больших текстов. На самом деле, в любом случае bottleneck-ом будет обучение модели, поэтому потерпим.

По своей сути этот подход является неким средним между двумя рассмотренными ранее методами, сохраняя самые лучшие их свойства.

Следующим шагом после получения токенизированной последовательности и формирования словаря необходимо получить векторные представления для каждого токена.

1.4. Эмбединги

Рассмотрим процесс обучения эмбедингов на примере подхода **Word2Vec**, где в первоначальной постановке токенами являются сами слова.

Пусть дана некоторая последовательность слов $w_1, \dots, w_T$. Для каждого слова w_i определим его контекст длины K как множество слов $\tilde{N}(w_i) = \{w_j : |i - j| \leq K, i \neq j\}$.

$$w_1, \underbrace{w_2, w_3, w_4, w_5, w_6}_{\text{context}}, w_7, w_8, \dots \quad i = 4, K = 2$$

Заведём две матрицы эмбедингов $U, V \in \mathbb{R}^{N \times d}$, где N – размер словаря, а d – размер эмбедингов. Вся разница лишь в том, что в матрице U лежат контекстные эмбединги токена, а в матрице V – центральные. Следовательно, каждому токenu $t_i \in \text{Vocab}$ соответствуют два эмбединга: один в качестве контекстного токена (i -ая строка матрицы U), другой в качестве центрального (i -ая строка матрицы V). Изначально U и V инициализированны случайно.

$$U = \begin{pmatrix} u_1 \\ \dots \\ u_i \\ \dots \\ u_N \end{pmatrix}, \quad V = \begin{pmatrix} v_1 \\ \dots \\ v_i \\ \dots \\ v_N \end{pmatrix}$$

$$t_i \in \text{Vocab} \Rightarrow \text{CentralEmbedding}(t_i) = u_i, \text{ContextEmbedding}(t_i) = v_i \in \mathbb{R}^d$$

Нашей целью является обучить эмбединги так, чтобы они содержали информацию о внутреннем устройстве языка. Для Word2Vec есть две такие задачи, рассмотрим их на примере контекста слова w_4 выше (в общем случае K может быть другим!):

- **Skip-gram.**

Модель обучается на предсказание вероятности встретить слово в контексте какого-то центрального слова:

$$\mathbb{P}(w_2 | w_4), \mathbb{P}(w_3 | w_4), \mathbb{P}(w_5 | w_4), \mathbb{P}(w_6 | w_4)$$

В качестве близости двух слов возьмем скалярное произведение соответствующих эмбедингов. Так как модель должна выдавать вектор вероятностей, в общем случае

$$\mathbb{P}(w | w_i) = \text{SoftMax} \left(\begin{bmatrix} \langle u_1, v_i \rangle \\ \dots \\ \langle u_j, v_i \rangle \\ \dots \\ \langle u_N, v_i \rangle \end{bmatrix} \right) = \begin{bmatrix} \mathbb{P}(w_1 | w_i) \\ \dots \\ \mathbb{P}(w_j | w_i) \\ \dots \\ \mathbb{P}(w_N | w_i) \end{bmatrix}$$

Теперь можно пройтись скользящим окном по всему тексту из обучаемой выборки и собрать большое количество объектов. То есть задача свелась к классификации:

$$\mathcal{L} = - \sum_{\substack{\text{center } w_j \\ \text{context } w_i}} \log \mathbb{P}(w_i | w_j) = - \sum_{\substack{\text{center } w_j \\ \text{context } w_i}} \log \left(\frac{\exp \langle u_i, v_j \rangle}{\sum_{k=1}^N \exp \langle u_k, v_j \rangle} \right) \rightarrow \min_{U, V}$$

• Continuous Bag of Words (CBOW).

Модель обучается на предсказание вероятности встретить слово в качестве центрального при условии какого-то контекста:

$$\mathbb{P}(w_4 | w_2, w_3, w_5, w_6)$$

В качестве эмбединга контекста размера K слова w_i тут выступает просто сумма

$$\text{Context}(w_i) = u_{i-K} + \dots + u_{i-1} + u_{i+1} + \dots + u_{i+K}$$

В остальном все как в skip-gram – теперь у нас есть один эмбединг контекста и один эмбединг центрального слова $\Rightarrow$ считаем скалярное произведение:

$$\begin{aligned} \mathbb{P}(w | w_{i-K}, \dots, w_{i-1}, w_{i+1}, \dots, w_{i+K}) = \\ = \text{SoftMax} \left(\begin{bmatrix} \langle v_1, \text{Context}(w_i) \rangle \\ \dots \\ \langle v_j, \text{Context}(w_i) \rangle \\ \dots \\ \langle v_N, \text{Context}(w_i) \rangle \end{bmatrix} \right) = \begin{bmatrix} \mathbb{P}(w_1 | w_{i-K}, \dots, w_{i-1}, w_{i+1}, \dots, w_{i+K}) \\ \dots \\ \mathbb{P}(w_j | w_{i-K}, \dots, w_{i-1}, w_{i+1}, \dots, w_{i+K}) \\ \dots \\ \mathbb{P}(w_N | w_{i-K}, \dots, w_{i-1}, w_{i+1}, \dots, w_{i+K}) \end{bmatrix} \end{aligned}$$

Дальнейшие действия идентичны skip-gram: проходимся окном по тексту, набираем объекты для обучения и решаем задачу классификации.

Однако возникает проблема: все это будет обучаться, но очень долго. При подсчете вероятностей и лосса необходимо считать суммы вида

$$\sum_{k=1}^N \exp \langle u_k, v_i \rangle,$$

то есть проходить по всему словарю. Хотелось бы считать такое без полного прохода.

Этого можно решить переходом от задачи многоклассовой классификации к бинарной:

$$\mathbb{P}(w_i \text{ из контекста } w_j) = \sigma(\langle u_i, v_j \rangle)$$

$$\mathcal{L} = - \sum_{\substack{\text{center } w_j \\ \text{context } w_i}} \log \mathbb{P}(w_i \text{ из контекста } w_j) = - \sum_{\substack{\text{center } w_j \\ \text{context } w_i}} \log \sigma(\langle u_i, v_j \rangle) \rightarrow \min_{U, V}$$

Утверждение. Теперь это вообще не будет учиться, ведь мы обучаем модель лишь на положительных парах, которые встречаются вместе (то есть только на положительном классе) $\Rightarrow$ ей выгодно обучить эмбединги так, чтобы скалярное произведение любых двух было максимально возможным.

Эта проблема решается довольно просто с помощью **negative sampling**: помимо положительных пар будем обучать модель и на отрицательных, то есть

$$\mathcal{L} = - \sum_{\substack{\text{center } w_j \\ \text{context } w_i}} \log \sigma(\langle u_i, v_j \rangle) - \sum_{\substack{w_i, w_j \\ \text{negative pair}}} \log (1 - \sigma(\langle u_i, v_j \rangle)) \longrightarrow \min_{U, V}$$

Есть множество способов семплировать отрицательные пары:

- В качестве отрицательной пары брать случайную пару слов – с большой вероятностью они не попадут друг к другу в контекст (при условии большого датасета)
- Можно делать сложные подсчеты статистик. Например, частоту встречаемости всех пар слов, чтобы потом при выборе отрицательной пары ориентироваться на это.

Говоря о балансе положительных и отрицательных пар, вторых как правило требуется сильно больше. В подобных задачах negative sampling выступает в роли регуляризации, то есть полученные эмбединги будут лучше обобщаться.

Для обучения таких матриц эмбедингов U и V в торче есть модуль `nn.Embedding`, который представляет из себя буквально то, что нужно: `nn.Embedding[i] = Embedding(t_i) \in \mathbb{R}^d`.

После обучения в качестве эмбедингов можно взять одну из матриц U , V или $\frac{1}{2}(U + V)$.

Магия заключается в том, что обучаясь лишь на какие-то статистические признаки в обычных текстах, модель выучивает внутреннюю семантику языка, хотя напрямую этому не училась!

$$v_{\text{man}} - v_{\text{king}} \approx v_{\text{woman}} - v_{\text{queen}}$$

Считается, что CBOW дает хорошие эмбединги для часто встречающихся слов, тогда как skip-gram – для редко встречающихся. Однако в целом Word2Vec является довольно древней вещью и с ним есть проблема – невозможность получения эмбединга новых слов.

1.5. FastText

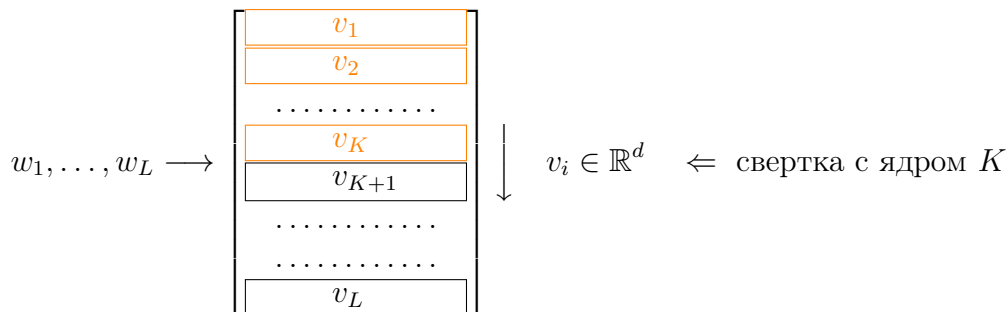
FastText улучшает подход skip-gram. Идея в том, чтобы оптимизировать вероятности не только для самих слов, но и для их n -грамм:

$$\begin{aligned} \mathbb{P}(\text{дрова} \mid \text{трава}) &\longrightarrow \max && \Leftarrow \text{skip-gram} \\ \text{трава} &\longrightarrow \{\text{тр, тра, рав, ава, ва}\} \\ \mathbb{P}(\text{дрова} \mid \text{тр}), \dots, \mathbb{P}(\text{дрова} \mid \text{ва}) &\longrightarrow \max && \Leftarrow \text{FastText} \end{aligned}$$

- (+) Возможность построить эмбединги для новых слов.
- (+) Обмен информации между разными словами (так как содержат одинаковые n -граммы).

1.6. TextCNN

TextCNN использует 1D свертки для обработки последовательностей. Сначала последовательность преобразуется в последовательность эмбедингов, а над ней применяется свертка:



Таким образом, из входа размера $d_{\text{in}} \times L_{\text{in}}$ свертка выдает $d_{\text{out}} \times L_{\text{out}}$, где как и в картинках d_{out} является гиперпараметром, а L_{out} зависит от паддинга, страйда и так далее.

Проблема в том, что для обучения батчами последовательности должны быть одинаковой длины, а это не так. Для этого вводится специальный токен $\langle \text{PAD} \rangle$ (обычно нулевой вектор), с помощью которого все последовательности эмбедингов добиваются до длины самой длинной последовательности в батче.

При этом в CNN обычно перед линейным слоем делается какой-нибудь AveragePooling, но в случае TextCNN этот пулинг должен быть с учетом длин последовательностей внутри батча, не учитывая паддинг, иначе получится что-то странное.